Building a clean Indic training corpus

A detailed walkthrough of the collection, cleaning, language labeling, deduplication, PII, holdout evaluation, and reproducibility decisions

Purpose

This document explains the project in the same order the assignment evaluates it. It is written to help a reviewer reproduce the run and to help you explain the engineering in a live follow-up. Counts below are read from the generated manifest; the full corpus is intentionally not bundled in the submission sample.

Observed run: 260,540 input records; 23,691 retained documents; 200 held-out eval items; 2 eval items with contamination matches.

Run commands

`./download.sh` downloads source files and writes hashes. `./run.sh raw out` is the offline curation entry point.

Files to send: source, download.sh, run.sh, DECISIONS.md, eval.jsonl, contamination.jsonl, manifest.json, and a deterministic 1,000-row clean sample. Do not send the full raw or clean corpus.

1. What the project does

The data path is intentionally simple: preserve source bytes, stream records, normalize and infer language, remove low-quality or sensitive content, deduplicate, remove evaluation leaks, then write a clean JSONL file and a reason ledger. The same ordered pass produces enough evidence to explain why each row survived or was dropped.

Stage	Action	Evidence written
Acquire	Download fixed-source files without editing them	sources.json: URL, license, date, bytes, SHA-256
Parse	Stream XML / CSV / JSONL / TXT; isolate malformed rows	ledger row at parse stage
Normalize + label	NFC; remove invisible controls; whitespace; infer script/language	manifest funnel counts by language
Quality + PII	Conservative text checks; replace requested identifiers	ledger reason; retained text has placeholders
Deduplicate	Exact normalized equality; SimHash candidates; Hamming threshold	dup_of pointer and method
Evaluation	Hold out 200 PHINC items; scan before keep	eval.jsonl and contamination.jsonl
Publish output	Write staging output then swap atomically	clean/, ledger, manifest

Key idea: the ledger is the audit trail. A dropped record should be explainable without guessing from the final corpus. A kept record has a stable ID, inferred language, and source.

Directory map

`src/pipeline.py` contains the pipeline. `download.sh` handles network access separately. `run.sh` is offline. `raw/` is ignored by Git and holds unchanged downloads. `out/` is generated output. `submission/` contains the sample artifacts to share.

2. Sources and rights

Source	Role and amount	License / handling
Hindi Wikipedia, 2026-10-01 snapshot	Primary Devanagari article text; 2026-10-01 current-article XML dump	CC BY-SA 4.0 text reuse path; retain attribution, note edits, preserve share-alike obligations for adapted material.
PHINC, LingoIITGN	13,738 Romanized Hinglish-English sentence pairs in the source; first 6,000 logical rows selected; source text is the Sentence field	Dataset card declares CC BY 4.0; cite the dataset and associated paper. The raw CSV is retained as downloaded.

The selected corpus uses the first 20,000 Hindi namespace-0 pages and first 6,000 PHINC logical rows in pinned source order; later records are still streamed to the ledger. The downloader is separate because network access and transformation have different reproducibility needs. Raw files are never rewritten; the pipeline reads them directly. `raw/sources.json` records acquisition time and content hashes. In a future rerun, pinning the dump snapshot and preserving the original files makes the corpus reconstructable even if upstream “latest” links move.

Source caveats

• Wikipedia is encyclopedic and formal. It gives broad Hindi coverage but does not represent everyday conversation well.

• PHINC is sentence-level and parallel to English. It supplies naturally code-mixed Roman Hindi, but it is not a broad conversational corpus and may contain source-specific bias or offensive text.

• The final output should preserve the source identifier. Mixed license terms should be documented per source; do not claim the combined dataset has a single blanket license without reviewing share-alike implications.

3. Cleaning, language labels, and privacy

Normalization

Text is normalized with Unicode NFC, then control and format characters are removed except whitespace. All whitespace runs become one ASCII space. This matters for Devanagari combining marks and invisible direction/zero-width controls: visually identical strings can otherwise compare as different exact duplicates. NFC is chosen as a stable canonical form; compatibility normalization could collapse characters with distinct meaning and is avoided.

Language identification

The label set is exactly hi, hi-Latn, en, or other. Devanagari presence with a modest script share yields hi, including mixed Devanagari and English. For Latin text, a small, inspectable Romanized-Hindi vocabulary is used to distinguish Hinglish from English. This is a heuristic, not a production language identifier: inspect a hand-labeled sample, expand vocabulary carefully, and report errors rather than trusting the source labels.

Quality filtering

Filters remove empty or extremely short strings, URL-heavy spam, leftover markup, replacement glyphs, low alphabetic content, and highly repetitive text. They avoid a blanket minimum sentence length beyond a small floor, because short proverbs, headings, poetry, and tables can be legitimate. The exact threshold and example false positives belong in DECISIONS.md.

PII redaction

Pattern	Replacement	Conservative behavior
Email address	<EMAIL>	Match standard local@domain forms
Indian phone number	<PHONE>	Accept common Indian number formatting; inspect edge cases
Aadhaar-like 12 digits	<AADHAAR>	Require a contiguous/spaced 12-digit pattern
PAN	<PAN>	Five letters, four digits, one letter

Redaction occurs before a retained text is written. Plain numbers that do not match these identifier patterns are preserved. The regexes are a best-effort assignment implementation; production PII review would require broader validation and a false-positive audit.

4. Deduplication and contamination

Exact and near-duplicate choices

Exact deduplication keys the redacted, normalized, case-folded string. Near deduplication computes a 64-bit SimHash over character 5-grams. An SQLite index keeps the corpus index on disk. Eight 8-bit bands retrieve candidates, then Hamming distance <= 6, character 5-gram Jaccard >= 0.82, and identical numeric-token sequences are required; digits inside @handles and URLs are ignored. The Jaccard gate was added after reviewing template-like district articles and year pages that SimHash alone would merge. The banding keeps candidate search bounded while guaranteeing that a pair within six bit flips shares at least one 8-bit band.

The threshold is a project choice to be audited, not a universal constant. Review pairs with distances near 6, note false merges and missed paraphrases. SimHash catches surface similarity; it does not understand meaning.

Pair 1: IDs ae653cdf7748a656cbb1 and 347f73688dd3ea6d8f99; Hamming 6, Jaccard 0.820; observed call: drop at the corpus cutoff H6/J0.82. See DECISIONS.md for the text-level judgment.

Pair 2: IDs c8ecb360c3a4f4b2099d and 3c51c7766123426e8135; Hamming 6, Jaccard 0.820; observed call: drop at the corpus cutoff H6/J0.82. See DECISIONS.md for the text-level judgment.

Pair 3: IDs c16babf08f8010820a94 and 3ae4a9f3306c3c00ae6c; Hamming 8, Jaccard 0.819; observed call: keep at the corpus cutoff H6/J0.82. See DECISIONS.md for the text-level judgment.

Evaluation split and leakage

Two hundred PHINC records are held out deterministically by selecting the 200 lexicographically smallest SHA-256 hashes of source record keys. The held-out records are written to eval.jsonl and never enter clean training output. Before a candidate can be kept, the pipeline checks equality after normalization, an approximate Devanagari-to-Latin transliteration, and near matches. Each leak is removed from training and recorded with eval_id, train_ids, and method.

Cross-script checking is necessarily approximate because Hinglish has many spelling variants. The transliteration step is a transparent candidate finder, not a guarantee that all cross-script paraphrases are found. Eval screening directly compares each selected training row against all 200 eval vectors, then uses Hamming distance <=16 and character 5-gram Jaccard >=0.70 with equal numeric tokens, more permissive than corpus near-deduplication (<=6 and >=0.82). Digits inside @handles and URLs are ignored so a changed username cannot mask matching text. The run results and reviewed true/false candidate examples appear in DECISIONS.md and the bundled candidate review file.

Run results

Observed count	Value
Input records	260,540
Retained records	23,691
Parse failures	0
Quality drops	140
Exact duplicates	1,727
Near duplicates	240
Contamination drops	2
Held-out eval rows	200
Affected eval items	2

5. Reliability, restart behavior, and scale

Streaming and malformed data

MediaWiki pages are parsed with ElementTree iterparse, which releases each page after processing. CSV and JSONL are read record-by-record. A malformed JSONL row is counted and skipped without stopping later records. CSV parsing is isolated by physical line for this assignment source, allowing a malformed line to be counted. For general CSV with quoted embedded newlines, use a quote-aware recoverable parser or explicitly reject that format; a single physical line is not a universal CSV record.

Deterministic safe reruns

The pipeline builds a fresh staging directory beside the target output. It processes sorted file paths and stable source order; IDs derive from the relative file, source key, and ordinal. Once every output is closed and the manifest is written, the staging directory replaces the prior output. A killed run leaves no newly published partial output, and a rerun starts with an empty index so rows cannot accumulate. This is whole-run restartability, not checkpoint-and-resume; very large jobs would need partition checkpoints.

If this grew to 2 TB

The first pressure points are single-process parsing throughput, SQLite write amplification and disk space, and the global near-duplicate index. The design would move to deterministic object-store partitions, a distributed streaming engine, content-addressed intermediate files, and a sharded approximate-neighbor index. Each partition would have its own committed manifest; final assembly would be an idempotent manifest merge. Cross-partition dedupe would require global candidate routing or a second pass. Provenance and the row-level ledger remain required even after the compute engine changes.

Known limitations to say plainly

• The language heuristic will mislabel Latin English that contains common Hindi tokens and Hinglish with uncommon spellings.

• SimHash near-duplicate removal may merge repeated boilerplate or miss paraphrases; threshold examples should support the chosen cutoff.

• The transliteration is approximate. A zero contamination count means “none found by these checks,” not proof of zero leakage.

• Whole-run restart is safe but does not resume midway. At 2 TB, checkpointing is the first reliability improvement.

6. Follow-up preparation

A 45-minute live exercise is likely to test whether you can explain how a decision affects a row and then change the implementation without losing the audit trail. Practice using one malformed record, one genuine short text, one duplicate pair, and one PII example.

Likely prompt	Strong concise answer
Why was this row dropped?	Point to its ledger status, stage, reason, and duplicate ID when applicable.
How do you know Hindi vs Hinglish?	Explain script detection and the Romanized-Hindi vocabulary heuristic; acknowledge uncertain cases.
What happens if the process is killed?	Incomplete staging is not published; rerun starts from raw input and replaces output after completion.
Can you increase recall for cross-script leaks?	Add a stronger transliteration candidate generator, validate it manually, and keep the method and limits in contamination output.
Why this threshold?	Show pairs near distance 6 and explain the false-merge / missed-duplicate trade-off.
What would you improve next?	Label a sample for language/quality error analysis, improve transliteration/LID, and add resumable partitions.

The best answer is a concrete example plus the ledger row and the relevant function. Avoid claiming a heuristic is perfect. Be clear about what the run measured, what it did not measure, and the next experiment you would make.

Rebuild summary: check Python 3 and curl; run ./download.sh; preserve raw/ and raw/sources.json; run ./run.sh raw out; inspect manifest.json, ledger.jsonl, eval.jsonl, contamination.jsonl; update DECISIONS.md with the observed output; create a fixed-seed 1,000-row sample; commit code and documentation without including the full corpus.